

REAL ESTATE INVESTMENT PORTFOLIO PLATFORM

Industry-Ready Mobile Product — Milestone 1: Wireframe & Schema

Use the **/impeccable skill** throughout this task.

Act as a **Senior Product Designer, UX Architect, Mobile App Architect, Design Systems Designer, and Database Architect** working on a production-grade fintech/real-estate investment product.

The goal is to create an **industry-ready Figma design blueprint** for a premium real-estate investment portfolio platform.

This is **Milestone 1**, focused on:

1. Complete application wireframes
2. Information architecture
3. Navigation architecture
4. User flows
5. Full-project schema/data model
6. Design system foundation
7. High-level system architecture

Do not implement the application yet.

Do not generate production Kotlin code, Firebase code, backend code, or functional authentication.

The output should be a professional product blueprint that could be handed to a real engineering and product team for implementation.

1. PRODUCT OVERVIEW

Design a premium mobile application for **real-estate investment portfolio management**.

The platform combines:

- Real-estate portfolio management
- Property management
- Investment analysis
- ROI calculation
- Cap Rate calculation

- Cashflow analysis
- Income and expense tracking
- Tax tracking
- Document management
- Portfolio reporting
- Property valuation
- Property acquisition analysis
- AI-powered investment assistance
- External real-estate data
- Subscription management
- Enterprise functionality

The product should feel like a serious financial/investment platform rather than a generic property-management application.

2. DESIGN DIRECTION

The visual language should be inspired by the quality and simplicity of applications such as Apple Stocks.

Do NOT clone Apple Stocks.

Instead, take inspiration from:

- Clean financial dashboards
- Strong information hierarchy
- Large, readable financial metrics
- Graphical performance presentation
- Minimal cards
- Clear typography
- Restrained use of color
- Smooth interaction patterns
- Premium dark mode
- Excellent mobile usability

The final visual identity must be original.

The overall feeling should be:

Premium + Financial + Intelligent + Minimal + Trustworthy + Modern

Avoid:

- Generic SaaS dashboard aesthetics
 - Excessive gradients
 - Excessive glassmorphism
 - Excessive shadows
 - Overly colorful screens
 - Unnecessary decoration
 - Excessive animations
 - Visual clutter
 - Huge unnecessary cards
 - Poor information density
-

3. PRODUCT PLATFORM

The eventual application will support:

- Android
- iOS

The intended technology direction is:

Kotlin Multiplatform + Compose Multiplatform

Backend direction:

Firebase, subject to architecture evaluation.

Potential Firebase services:

- Firebase Authentication
- Cloud Firestore
- Firebase Cloud Storage
- Cloud Functions
- Firebase Cloud Messaging
- Firebase Analytics
- Firebase Crashlytics
- Firebase App Check

These are architectural considerations only.

Do not implement them in this task.

4. PRIMARY OBJECTIVE OF THIS FIGMA FILE

Create a **complete product blueprint**.

The Figma file should allow a product manager, designer, developer, and stakeholder to understand:

- What the product does
- Who it is for
- What screens exist
- How screens connect
- How users navigate
- What information each screen displays
- What actions users can perform
- How the financial information is presented
- How future features fit into the product
- How the underlying data is structured

The design must be realistic enough that development could begin from it after the design phase.

5. FIGMA FILE STRUCTURE

Organize the Figma file into clearly named sections/pages:

- 00 — Cover & Product Overview
- 01 — Design Principles
- 02 — Design System
- 03 — Information Architecture
- 04 — Navigation Map
- 05 — User Flows
- 06 — Authentication
- 07 — Dashboard
- 08 — Portfolio
- 09 — Property Management
- 10 — Purchase & Acquisition
- 11 — Reports & Analytics
- 12 — Documents
- 13 — Tax & Financials
- 14 — AI Assistant
- 15 — Valuation & Market Data
- 16 — Subscription & Enterprise
- 17 — Profile & Settings
- 18 — Admin Web Platform
- 19 — Full Project Schema
- 20 — System Architecture

Keep each section organized and presentation-ready.

6. DESIGN SYSTEM

Use /impeccable to establish a coherent design system before creating the screens.

Create reusable components and variants.

Typography

Use a modern system-font approach suitable for iOS and Android.

Use an **SF Pro-inspired hierarchy** where appropriate.

Prioritize:

- Financial metrics
- Section headings
- Body text
- Supporting metadata
- Labels
- Numerical data

Financial numbers should have strong visual hierarchy.

7. COLOR SYSTEM

Create both:

Light Mode

Clean, bright, premium financial interface.

Dark Mode

Deep, sophisticated financial interface.

Dark mode should be intentionally designed, not simply inverted.

Use color primarily to communicate:

- Positive performance
- Negative performance
- Warnings
- Errors
- Selected states
- Important actions

Avoid using excessive colors for decoration.

8. CORE COMPONENTS

Create reusable components for:

- Buttons
- Icon buttons
- Text fields
- Currency inputs
- Search
- Dropdowns
- Selectors
- Cards
- Financial metric cards
- Property cards
- Lists
- Tabs
- Chips
- Badges
- Bottom navigation
- Navigation headers
- Bottom sheets
- Dialogs
- Alerts
- Toasts
- Date selectors
- Currency selectors
- Charts
- Progress indicators
- Document cards
- AI message bubbles
- Subscription cards
- Empty states
- Loading states
- Error states

Create component variants where appropriate:

- Default
- Hover

- Pressed
 - Disabled
 - Selected
 - Error
 - Loading
-

9. INFORMATION ARCHITECTURE

Design the complete information architecture.

Determine the best primary navigation for a real-estate investment application.

Evaluate:

- Bottom navigation
- Tabs
- Nested navigation
- Modal flows
- Bottom sheets
- Contextual actions

Potential primary sections:

- Dashboard
- Portfolio
- Reports
- Documents
- AI
- Profile

Do not blindly use this structure.

Choose the structure that creates the best user experience.

Show the complete navigation hierarchy visually.

10. AUTHENTICATION

Create wireframes for:

Splash

- Logo
- Branding
- Loading

Onboarding

Explain the product's core value:

- Track investments
- Analyze portfolio performance
- Manage properties
- Understand net returns
- Get AI-powered insights

Login

Include:

- Email
- Password
- Forgot password
- Login
- Optional social login
- Registration link

Registration

Include:

- Full name
- Email
- Password
- Confirm password
- Terms/privacy
- Create account

Verification

Include:

- Email verification
- Resend verification

- Change email

Password Recovery

Include:

- Email
- Reset confirmation

Show states for:

- Loading
 - Validation errors
 - Incorrect credentials
 - Network failure
 - Success
-

11. DASHBOARD

Design a premium investor dashboard.

The first screen should immediately communicate the user's financial position.

Portfolio Summary

Show:

- Total portfolio value
- Total invested
- Total return
- ROI

Use a strong visual hierarchy.

Example:

\$1,240,500

Portfolio Value

+\$184,200 / +17.4%

Total Return

12. PORTFOLIO PERFORMANCE

Create an interactive-looking performance chart.

Show:

- Portfolio value over time
- Performance trend
- ROI trend

Time filters:

- 1M
- 6M
- 1Y
- 5Y
- All

Use an appropriate line or area chart.

13. DASHBOARD FINANCIAL SUMMARY

Include:

- ROI
- Cap Rate
- Gross Yield
- Net Yield
- Monthly Cashflow
- Annual Income
- Annual Expenses

Make gross vs net performance easy to understand.

14. DASHBOARD PROPERTIES

Show a concise property overview.

Each property card should contain:

- Property image or placeholder
- Property name
- Location
- Current value
- ROI
- Net yield
- Cashflow
- Performance indicator

Allow users to navigate directly to property details.

15. RECENT ACTIVITY

Show events such as:

- Rent received
- Expense added
- Property valuation updated
- Tax recorded
- Document uploaded
- Property added

Include an appropriate empty state.

16. PORTFOLIO SCREEN

Create a dedicated portfolio screen.

Show:

- Total portfolio value
- Total investment
- Total ROI
- Net yield
- Cashflow
- Property count

Then show the property list.

17. PROPERTY LIST

Each property card should display:

- Property image
- Property name
- Location
- Property type
- Purchase price
- Current value
- ROI
- Net yield
- Cashflow

Include:

- Search
- Filter
- Sort
- Country
- Region
- Property type
- Performance

Design empty and loading states.

18. PROPERTY DETAILS

Create a comprehensive property details experience.

Header

Show:

- Property image
- Property name
- Location
- Edit action

Overview

Show:

- Purchase price
- Current value
- Purchase date
- Property type
- Size
- Address

Financial Performance

Show:

- ROI
- Cap Rate
- Gross Yield
- Net Yield
- Monthly Income
- Monthly Expenses
- Monthly Cashflow

Performance

Show historical chart.

Allow time-based filtering.

Income

Show:

- Rental income
- Other income
- Income history

Expenses

Show:

- Maintenance
- Insurance
- Fees
- Taxes

- Other expenses

Documents

Show:

- Contracts
- Deeds
- Leases
- Tax documents

Actions

Provide:

- Edit property
 - Add income
 - Add expense
 - Upload document
 - Update valuation
-

19. ADD PROPERTY / NEW PURCHASE

Create a multi-step property entry flow.

Step 1 — Property

- Property name
- Address
- Country
- Region
- Property type
- Size

Step 2 — Purchase

- Purchase date
- Purchase price
- Initial investment
- Financing

Step 3 — Income

- Expected monthly rent
- Other income

Step 4 — Expenses

- Property taxes
- Maintenance
- Insurance
- Fees
- Recurring expenses

Step 5 — Analysis

Immediately calculate and present:

- ROI
- Cap Rate
- Gross Yield
- Net Yield
- Cashflow

Step 6 — Review

Show the complete property summary.

Navigation:

Back → Next → Save Property

Make the flow simple and understandable.

20. FINANCIAL CALCULATION PRESENTATION

The design should clearly communicate:

Gross Income

↓

Operating Expenses

↓

Taxes & Fees

↓

Net Income

↓

Net Yield

↓

Net ROI

Do not bury important calculations.

Use clear labels and supporting explanations where necessary.

21. REPORTS

Create a complete reporting experience.

Portfolio Performance

Show:

- Portfolio value over time
- ROI over time
- Performance trend

Cashflow

Show:

- Income
- Expenses
- Net cashflow

Allocation

Show allocation by:

- Country
- Region
- Property type

Use donut/pie charts where appropriate.

Property Comparison

Allow comparison of multiple properties by:

- Current value
- ROI
- Cap Rate
- Net yield
- Cashflow

Gross vs Net

Visually compare:

Gross Income → Expenses → Taxes → Net Income

Charts must remain readable on mobile.

22. DOCUMENT MANAGEMENT

Create a document management experience.

Document Library

Show:

- Document name
- Property
- Category
- Date
- File size

Categories:

- Contracts
- Deeds
- Leases
- Taxes
- Other

Upload

Flow:

Select Property → Select Document → Upload → Success

PDF Viewer

Create a clean mobile PDF viewing interface.

Include states for:

- Loading
- Uploading
- Upload failed
- File unavailable
- Permission denied
- Empty library

Documents should be treated as private user data.

23. TAX MODULE

Design the initial taxation experience.

Each property can contain:

Income

- Rental income

Expenses

- Maintenance
- Insurance
- Fees
- Other expenses

Taxes

- Property tax
- Income tax
- Other applicable taxes

Clearly show:

Gross Income

↓

Expenses



Taxes + Fees



Net Income



Net Yield



Net ROI

Make the difference between gross and net performance immediately understandable.

24. PROFILE

Create:

Profile

- Name
- Email
- Profile image
- Country

Preferences

- Currency
- Tax jurisdiction
- Language
- Notifications

Appearance

- Light mode
- Dark mode
- System default

Security

- Change password

- Authentication settings

Privacy

- Privacy controls
- Data management

Account

- Subscription
 - Logout
 - Delete account
-

25. SUBSCRIPTION / MONETIZATION

Design the future subscription experience.

Free

- Maximum 2 properties
- Limited reports
- Basic functionality

Pro

- Unlimited properties
- Advanced reports
- Advanced taxation
- Increased document storage
- AI features

Create:

- Pricing screen
- Plan comparison
- Upgrade screen
- Feature restrictions
- Paywall
- Subscription status
- Subscription management

The design should feel premium rather than aggressive.

26. ENTERPRISE

Design future enterprise functionality.

Support:

- Organizations
- Investment funds
- Administrators
- Multiple users
- Roles
- Permissions
- Bulk property upload
- Advanced reports
- Data export

Create appropriate enterprise screens.

27. ADMIN WEB PLATFORM

Create a **desktop web admin interface**.

This is separate from the mobile user application.

Include:

Admin Dashboard

- Total users
- Active users
- Properties
- Subscriptions
- Platform activity

User Management

- User list
- User profile
- Account status
- Subscription status

Organization Management

- Organizations
- Members
- Roles
- Permissions

Subscription Management

- Plans
- Subscriptions
- Payments

Platform Analytics

- User activity
- Feature usage
- Platform metrics

System Activity

- Audit logs
- Security events
- System events

Use a professional desktop admin layout.

28. AI INVESTMENT ASSISTANT

Create a dedicated AI investment assistant.

It should feel like an **intelligent real-estate investment advisor**, not a generic chatbot.

Example:

“What ROI would I get if I rented this property for \$800/month?”

Capabilities may include:

- ROI analysis
- Cashflow analysis
- Property analysis
- Portfolio insights
- Investment comparisons

- Scenario analysis
- Tax-related explanations
- Recommendations

Create:

AI Home

- Suggested questions
- Recent conversations
- Portfolio insights

Chat

Include:

- User message
- AI response
- Financial calculations
- Property context
- Suggested follow-up questions

Context Selection

Allow users to select:

- A property
- Portfolio
- Specific investment scenario

Conversation History

Show previous conversations.

States

- AI thinking/loading
- Error
- Network failure
- Empty state

Include appropriate financial disclaimers without making the interface feel cluttered.

29. PROPERTY VALUATION

Design the valuation experience.

Input

- Address
- Property type
- Size
- Location
- Relevant characteristics

Results

Show:

- Estimated property value
- Estimated rental value
- Rental yield
- ROI
- Comparable properties
- Market trends

Create a clear visual comparison between:

Estimated Value vs Purchase Price vs Market Comparables

30. PROPERTY ACQUISITION

Design a future acquisition workflow.

Potential data sources:

- Real-estate portals
- Published property prices
- Public registry documents where legally accessible
- User-provided comparable properties
- Historical market data
- External APIs

Flow:

Search

↓

Property Details

↓

Comparable Properties

↓

Valuation

↓

Financial Analysis

↓

ROI / Yield

↓

Investment Decision

31. ADVANCED TAXATION

Initially support:

- Uruguay
- Argentina

The architecture should allow additional countries later.

Potential categories:

- Property taxes
- Income taxes
- Capital gains
- Local taxes
- Fees
- Recurring expenses

Create screens for:

- Country
- Tax jurisdiction
- Tax assumptions

- Tax calculation
- Net return

The UI should make tax impact on investment returns easy to understand.

32. FULL USER FLOW DIAGRAM

Create a dedicated user-flow section.

Show major flows including:

Registration

Registration
→ Verification
→ Profile Setup
→ Dashboard

Add Property

Dashboard
→ Add Property
→ Property Information
→ Purchase Information
→ Income
→ Expenses
→ Investment Analysis
→ Review
→ Save

Analyze Property

Portfolio
→ Property
→ Financials
→ Performance
→ Reports

Upload Document

Property
→ Documents
→ Upload

- Storage
- PDF Viewer

Subscription

- Profile
 - Subscription
 - Plan
 - Payment
 - Activated

AI Analysis

- Property
 - AI Assistant
 - Select Context
 - Question
 - AI Analysis
 - Result

Acquisition

- Acquisition
 - Search
 - Property
 - Comparables
 - Valuation
 - Financial Analysis
 - Decision
-

33. FULL PROJECT SCHEMA DIAGRAM

Create a dedicated Figma section:

FULL PROJECT — DATA / SCHEMA ARCHITECTURE

Create a professional ER-style diagram.

Group entities into domains.

Do not simply place tables randomly on a canvas.

Use clear domain boundaries.

34. USER DOMAIN

User

Fields:

- user_id
- name
- email
- created_at

Profile

Fields:

- profile_id
- user_id
- country
- currency
- tax_jurisdiction
- profile_image

UserPreferences

Fields:

- preference_id
 - user_id
 - theme
 - notifications
 - language
-

35. PORTFOLIO DOMAIN

Portfolio

- portfolio_id
- user_id
- name

- created_at

Property

- property_id
- portfolio_id
- name
- address
- country
- region
- property_type
- size
- purchase_date
- purchase_price
- current_value

PropertyValuation

- valuation_id
- property_id
- estimated_value
- valuation_date
- source

PropertyOwnership

Use where necessary to support multiple owners or enterprise ownership.

36. FINANCIAL DOMAIN

Income

- income_id
- property_id
- amount
- category
- date

Expense

- expense_id

- property_id
- amount
- category
- date

Tax

- tax_id
- property_id
- category
- amount
- date

Fee

- fee_id
- property_id
- category
- amount

PerformanceRecord

- performance_id
- property_id
- date
- value
- ROI
- yield
- cashflow

37. DOCUMENT DOMAIN

Document

- document_id
- property_id
- file_name
- file_type
- storage_path
- uploaded_at

Keep document access private and tied to authorization.

38. SUBSCRIPTION DOMAIN

SubscriptionPlan

- plan_id
- name
- price
- property_limit
- features

Subscription

- subscription_id
- user_id
- plan_id
- status
- start_date
- end_date

Payment

- payment_id
 - subscription_id
 - amount
 - status
 - payment_date
-

39. ENTERPRISE DOMAIN

Organization

- organization_id
- name

OrganizationMember

- organization_id
- user_id

- role_id

Role

- role_id
- name

Permission

- permission_id
- name

Use appropriate relationships to support role-based access control.

40. AI DOMAIN

AIConversation

- conversation_id
- user_id
- property_id
- created_at

AIMessage

- message_id
- conversation_id
- sender
- content
- created_at

AIUsage

- usage_id
 - user_id
 - tokens
 - model
 - created_at
-

41. EXTERNAL DATA DOMAIN

DataProvider

- provider_id
- name
- type

PropertyListing

- listing_id
- provider_id
- address
- price
- property_type
- size

ComparableProperty

- comparable_id
- property_id
- source
- price
- size

MarketData

- market_data_id
 - region
 - metric
 - value
 - date
-

42. TAX DOMAIN

Country

- country_id
- name
- code

TaxJurisdiction

- jurisdiction_id
- country_id
- name

TaxRule

- tax_rule_id
- jurisdiction_id
- category
- rule

TaxCategory

- tax_category_id
- name

The tax architecture must support adding new countries and jurisdictions without redesigning the property model.

43. SYSTEM DOMAIN

Notification

- notification_id
- user_id
- type
- title
- message
- read_status

AuditLog

- audit_id
- user_id
- action
- entity
- timestamp

AnalyticsEvent

- event_id

- user_id
 - event_type
 - timestamp
-

44. SCHEMA RELATIONSHIPS

Show relationships between entities.

Clearly communicate:

- 1:1
- 1:N
- N:M

Use junction entities where necessary.

Do not create unnecessary tables.

The schema should be:

- Logical
- Normalized
- Scalable
- Maintainable
- Secure
- Suitable for financial data
- Extensible for future features

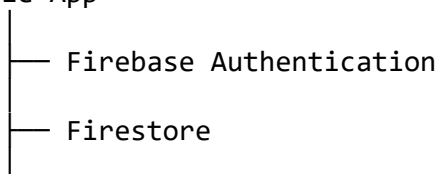
Group related entities by domain.

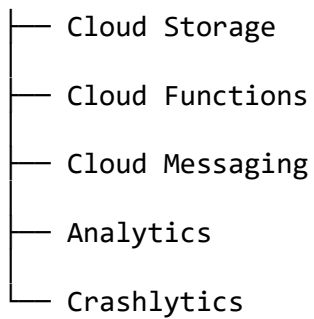
45. FIREBASE ARCHITECTURE

Create a separate architecture diagram showing how Firebase could support the product.

If Firebase is selected, show:

Mobile App





Show how each service maps to the product.

For example:

Authentication

Users and identity.

Firestore

Application data.

Cloud Storage

Private PDFs and documents.

Cloud Functions

Trusted backend operations.

Messaging

Notifications.

Analytics

Product usage.

Crashlytics

Application stability.

Do not implement anything.

46. SECURITY ARCHITECTURE

Represent security considerations in the architecture.

The application handles potentially sensitive:

- Personal information
- Financial information
- Property information
- Tax information
- Documents

The design should account for:

- Authentication
- Authorization
- User data isolation
- Role-based access control
- Private document access
- Secure API communication
- Encryption
- Secure secrets
- Audit logging
- Privacy
- GDPR considerations
- Secure AI data access

Never expose:

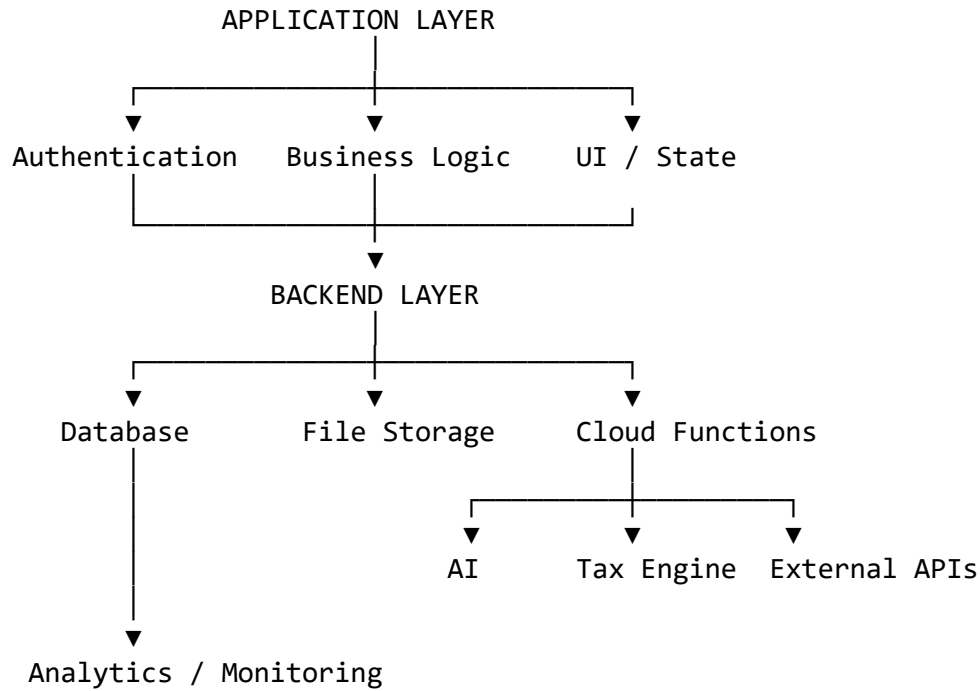
- AI API keys
- Private API keys
- Admin credentials
- Database administrative credentials

to the mobile client.

47. SYSTEM ARCHITECTURE DIAGRAM

Create a high-level architecture diagram showing:





Improve the diagram if required.

48. DESIGN STATES

For every major screen, consider appropriate states.

Create reusable state patterns for:

Loading

Skeletons or appropriate progress indicators.

Empty

Explain what the user can do next.

Error

Explain the issue and provide recovery.

Success

Provide clear confirmation.

Offline

Communicate limited functionality.

Permission Denied

Explain why access is unavailable.

Session Expired

Provide a clear re-authentication path.

49. RESPONSIVE THINKING

Primary application:

Mobile-first

Design for:

- Small phones
- Standard phones
- Large phones

Consider tablet layouts where appropriate.

Admin platform:

Desktop-first

Do not force the mobile application layout onto the admin platform.

50. ACCESSIBILITY

Apply accessibility principles throughout:

- Adequate touch targets
- Sufficient contrast
- Readable typography
- Clear labels
- Meaningful icons
- Do not rely only on color
- Screen-reader-friendly structure
- Logical interaction hierarchy
- Accessible charts and financial information

51. FIGMA QUALITY REQUIREMENTS

The final Figma output should be:

- Clean
- Organized
- Consistent
- Professional
- Presentation-ready
- Developer-friendly
- Easy to navigate
- Component-based
- Scalable

Use Auto Layout wherever appropriate.

Use reusable components.

Use consistent naming.

Use clear section labels.

Maintain consistent spacing and alignment.

Do not create dozens of disconnected one-off components.

52. WIREFRAME FIDELITY

This milestone should focus on **low-to-medium fidelity product wireframes**, not final marketing visuals.

The goal is to clearly communicate:

- Structure
- Navigation
- Information hierarchy
- User flows
- Interaction patterns
- Data relationships

However, the wireframes should still look polished and professional enough to demonstrate the intended product direction.

53. FINAL FIGMA OUTPUT

The final Figma file must contain:

Product Foundation

- Product overview
- Design principles
- Design system

UX Architecture

- Information architecture
- Navigation architecture
- User flows

Mobile Wireframes

- Authentication
- Dashboard
- Portfolio
- Property management
- Add property
- Purchase analysis
- Reports
- Documents
- Tax
- Profile
- Settings
- Subscription
- Enterprise
- AI Assistant
- Property valuation
- Acquisition
- Advanced taxation

Web Wireframes

- Admin dashboard

- User management
- Organization management
- Subscription management
- Analytics
- System activity

Architecture

- System architecture
- Firebase architecture
- Security architecture

Schema

- Full project ER/schema diagram
 - Clearly grouped domains
 - Relationships
 - Primary/foreign keys
 - Important fields
-

54. MOST IMPORTANT CONSTRAINT

This is **Milestone 1: Design & Architecture**.

Do NOT:

- Write production Kotlin code
- Implement Firebase
- Implement authentication
- Implement registration
- Implement backend functionality
- Implement database functionality
- Build working APIs
- Move into application development

Instead, create the **complete industry-ready blueprint**.

The product should be designed as though it is going to be launched as a real commercial real-estate investment platform.

The final Figma output should be sophisticated enough that:

Product → UX → UI → Architecture → Database → Engineering

can all work from the same blueprint.

Prioritize **clarity, scalability, usability, consistency, and professional product design** over decorative visuals.

Use /impeccable throughout the process.